Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

List of teammates

S.no	Name	College	Contact
1	Ranjan C	Kaamadhenu Arts and Science	ranjanc.cs23@kascsathy.ac.in
2	PUNITHAVATHI S	Kaamadhenu Arts and Science	punithavathi.cs23@kascsathy.ac.in
3	VISWANATHAN R	Kaamadhenu Arts and Science	viswanathan.cs23@kascsathy.ac.in
4	VIGNESHWARAN S	Kaamadhenu Arts and Science	vigneshwaran.cs23@kascsathy.ac.in

Abstract

Traditional VPN architectures often struggle to address modern cybersecurity challenges such as lateral movement and excessive network privileges. These limitations highlight the need for a more secure and adaptive model based on Zero Trust principles. This project focuses on the design, deployment, and management of a Zero Trust Network Access (ZTNA) framework using Zscaler Private Access (ZPA) within a large-scale institutional network environment.

The primary objective of the project is to replace conventional network-level access with a more secure application-specific and identity-based connectivity model. By deploying Zscaler App Connectors within the institutional data center and integrating the system with existing Identity Providers such as Azure Active Directory and endpoint security solutions, the architecture ensures that all connections between users and applications are securely brokered through a cloud-based platform.

This approach effectively hides internal applications from direct internet exposure, thereby minimizing potential attack surfaces. In addition, the project examines the integration of Zscaler with other security platforms including Security Information and Event Management (SIEM) tools such as QRadar for centralized logging and monitoring, and MISP for threat intelligence integration to support dynamic and adaptive access policies.

The implementation of this architecture results in a highly scalable and resilient security framework that enforces the principle of least-privilege access, enables comprehensive auditing capabilities, and significantly strengthens the institution's overall cybersecurity posture.

Use Case Scenarios

Scenario 1: Secure Remote Access for Faculty to ERP System

A faculty member working remotely from home needs to upload student grades through the university's ERP system. Instead of connecting through a traditional VPN, the user accesses the application using Zscaler Private Access (ZPA). The system first authenticates the user through Azure Active Directory with Multi-Factor Authentication (MFA) and validates the device security posture before allowing access.

Output:

- User identity successfully verified (Faculty_Group)
- Device posture validated (Antivirus active, operating system updated)
- Secure micro-tunnel established only to the ERP application
- Internal network resources remain hidden from the user

Scenario 2: External Researcher Accessing a Specific Database

An external research collaborator from another institution requires temporary access to a research database hosted on the university servers. The administrator creates a time-restricted, application-specific policy in Zscaler ZPA, ensuring that the researcher can only access the designated database without visibility to other internal systems.

Output:

- External researcher authenticated using temporary credentials
- Access policy enforced (Application: Research-DB, Duration: 7 days)
- Database server remains invisible to internet scanning tools
- User session automatically expires after the policy duration

---

Scenario 3: Student Access to Online Examination Portal

During university examinations, thousands of students simultaneously connect to the internal online examination portal from different locations. Zscaler ZPA ensures that only authenticated students using compliant devices can access the exam platform while preventing unauthorized access to other institutional resources.

Output:

- Large-scale user authentication supported through cloud scalability
- Students connected exclusively to the examination portal
- Unauthorized lateral movement to other servers blocked
- Exam server IP address remains concealed from external networks

---

Scenario 4: IT Administrator Performing Server Maintenance

An IT administrator needs to perform maintenance on a critical application server. Instead of gaining full network access through a VPN, the administrator connects securely through ZPA to the specific server management interface. The session is monitored continuously and any suspicious activity is automatically blocked.

Output:

- Administrator identity verified using Multi-Factor Authentication
- Secure micro-tunnel created only for the server management interface
- Administrative commands and keystrokes logged for auditing
- Unauthorized attempts to access other servers automatically blocked

---

Scenario 5: Temporary Contractor Access During System Upgrade

During a university system upgrade project, a contractor requires temporary access to a specific file server for data migration. Using Zscaler ZPA, the contractor is granted limited application-level access without being added to the organization's internal directory services or VPN infrastructure. The access is automatically revoked once the project timeline ends.

Output:

- Contractor authenticated through a separate Identity Provider (IdP)
- Access limited only to the File-Server-Migration resource
- All file transfer activities recorded in Zscaler and SIEM logs
- Access privileges automatically revoked after the 30-day project period

Stage – 1

Implement and manage Zero Trust Network Access (ZTNA) using Zscaler

Overview

The objective of this project is to implement a Zero Trust Network Access (ZTNA) architecture using Zscaler in order to enhance secure connectivity between users and internal applications. Traditional VPN-based access models provide broad network-level access once a user is authenticated, which can expose the internal network to risks such as unauthorized access, lateral movement, and cyber-attacks.

To overcome these limitations, the Zero Trust model follows the principle of "Never Trust, Always Verify." Every user, device, and application request must be authenticated and verified before access is granted. Instead of providing full network connectivity, Zscaler Private Access (ZPA) establishes secure, application-specific micro-tunnels between authenticated users and authorized applications.

The project also integrates AI-driven identity verification mechanisms that analyze user behavior patterns to strengthen security. Machine learning techniques such as deep learning, neural networks, and behavioral analytics are used to generate dynamic user profiles. These profiles help distinguish legitimate user behavior from suspicious activities in real time.

Real-time anomaly detection mechanisms continuously monitor user activities. If unusual behavior is detected, the system triggers additional authentication procedures using multi-factor authentication (MFA). Alerts are sent to both users and administrators to ensure quick response to potential threats.

The system includes secure data storage and management components to protect sensitive user identity data and behavioral profiles. Encryption techniques are used to secure data both in

transit and at rest. In addition, firewalls, intrusion detection systems, and secure authentication protocols are implemented to strengthen the overall security architecture.

An integration layer allows the system to connect with enterprise applications, monitoring platforms, and security tools. Monitoring and reporting features provide transparency and help administrators track system performance and security events.

The architecture is designed to be scalable and flexible, allowing deployment on cloud infrastructure when required. Load balancing and distributed deployment improve system performance and reliability. Continuous training and optimization of machine learning models ensure the system adapts to evolving user behavior and emerging cyber threats. Geographic distribution of system components further improves redundancy and availability.

List of Vulnerabilities

Practice Website: http://www.itsecgames.com/

S.No	Vulnerability Name	CWE No
1	Improper Sanitization of File Paths	CWE-284 – Improper Access Control
2	Invalid Certificate	CWE-395 – Improper Certificate Validation
3	Missing X-Frame-Options Header	CWE-1021 – Improper Restriction of Rendered UI Layers
4	X-Content-Type Options Header Missing	CWE-918 – Server-Side Request Forgery (SSRF)
5	Drupal Ajax API JSONP Not Disabled	CWE-937 / CWE-1035 – Using Components with Known Vulnerabilities
6	Drupal API Vulnerability	CWE-20 – Improper Input Validation
7	Improper API Filtration	CWE-79 – Cross-Site Scripting (XSS)
8	Denial of Service	CWE-400 – Uncontrolled Resource Consumption

9	TLS Version 1.0 Protocol Detection	CWE-327 – Use of Broken Cryptographic Algorithm
10	web.config File Information Disclosure	CWE-497 – Exposure of Sensitive System Information

Main Website Vulnerabilities

S.No	Vulnerability Name	CWE No
1	Medium Strength Cipher Suites Supported (SWEET32)	CWE-200 – Exposure of Sensitive Information
2	Cookie SERVERID created without Secure Flag	CWE-79 – Cross-Site Scripting
3	X-Content-Type Options Header Missing	CWE-918 – Server-Side Request Forgery
4	Denial of Service	CWE-400 – Uncontrolled Resource Consumption
5	OS Identification	CWE-200 – Exposure of Sensitive Information

---

Vulnerability Report

Practice Website: http://www.itsecgames.com/

1. Vulnerability Name: Improper Sanitization of File Paths
CWE: CWE-284 – Improper Access Control
OWASP Category: A01:2021 – Broken Access Control

Description

This vulnerability occurs when the application fails to properly validate or restrict access to file paths. Attackers may manipulate file paths in application requests to access sensitive files or directories that should normally be restricted from public access.

Business Impact

If exploited, this vulnerability can allow attackers to gain unauthorized access to sensitive system files, configuration data, or user information. This may result in data breaches, privacy violations, and damage to the organization's reputation.

Steps to Reproduce

1. Identify the vulnerable host using vulnerability scanning tools.
2. Access the host information using the following link: https://www.shodan.io/host/31.3.96.40
3. Attempt to manipulate file path parameters within application requests.
4. If restricted directories or files become accessible without proper authorization, the vulnerability is confirmed.

Mitigation

• Implement strict input validation and file path sanitization.
• Enforce proper access control mechanisms.
• Restrict directory traversal attempts.
• Apply security patches and secure configuration practices.

Recommendation

1. **Apply Security Updates:** The first and most important step is to apply the latest security updates released by Drupal. These updates usually contain patches for known vulnerabilities and help protect the system from potential attacks.
2. **Review Configuration:** After applying updates, the Drupal site configuration should be carefully reviewed to ensure that private files and sensitive resources remain properly protected. File permissions and access settings should be verified and adjusted if necessary.
3. **Regular Security Audits:** Organizations should conduct periodic security audits and vulnerability assessments to identify potential security weaknesses. Regular monitoring helps in detecting and resolving security issues at an early stage.

4. **Access Control Management:** Strong access control policies must be implemented to ensure that only authorized users can access private files and sensitive resources. File permissions should be reviewed regularly and restricted based on user roles.
5. **Data Minimization:** Only essential data should be stored within the Drupal system. Reducing the amount of sensitive data stored on the platform lowers the risk of exposure in case of a security breach.

---

2) Vulnerability Name: Invalid Certificate

CWE: CWE-395 – Improper Certificate Validation
OWASP Category: A07:2021 – Identification and Authentication Failures

Description

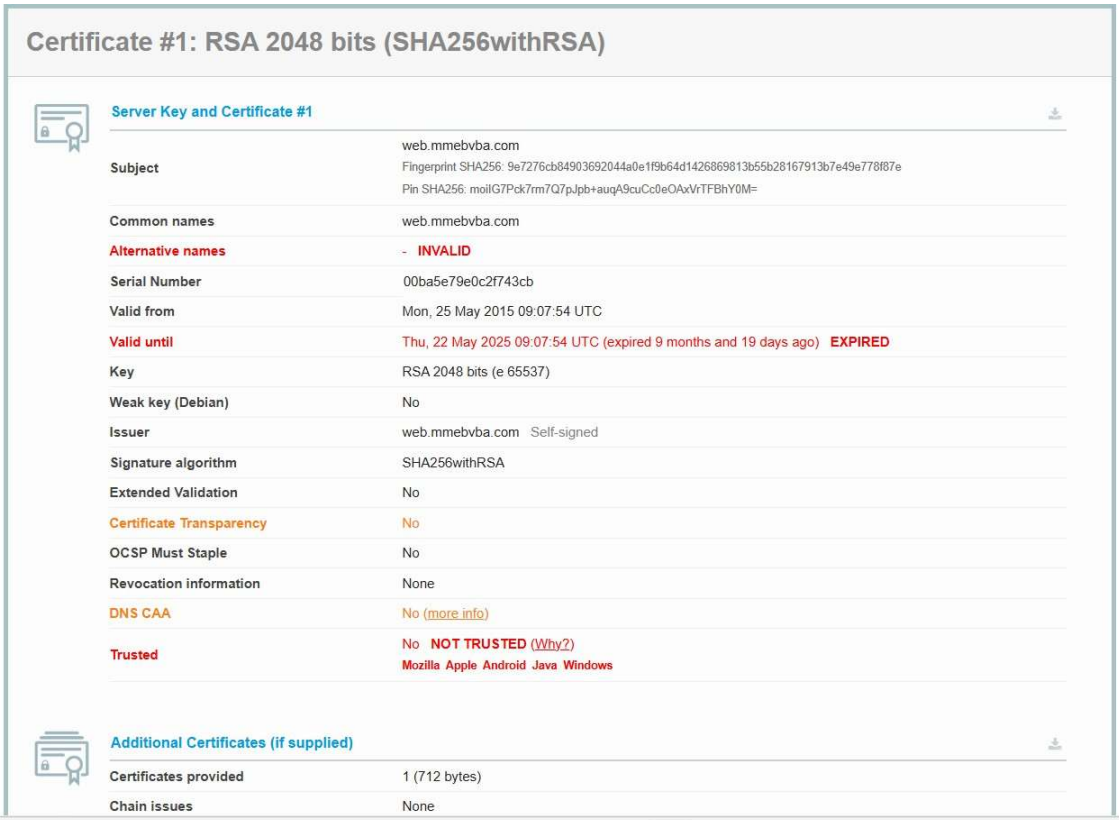
This vulnerability occurs when a system fails to properly validate digital certificates during secure communication. If the certificate validation process is weak or improperly implemented, attackers may exploit this weakness to impersonate a trusted server or service. As a result, the system may unknowingly establish a connection with a malicious host while assuming it is communicating with a legitimate one.

Business Impact

Improper certificate validation can lead to several serious security threats, including:

- **Man-in-the-Middle (MITM) Attacks:** Attackers may intercept communication between two parties and monitor or modify the exchanged data.
- **Data Breaches:** Sensitive information such as login credentials or confidential data may be exposed to unauthorized attackers.
- **Malware Distribution:** Attackers may use fake or compromised certificates to distribute malicious software or compromise user systems.
- **Loss of Trust:** Security breaches caused by improper validation can damage the organization's reputation and user confidence.

Steps to Reproduce: https://www.ssllabs.com/ssltest/analyze?d=www.itsecgames.com



Certificate #1: RSA 2048 bits (SHA256withRSA)

Server Key and Certificate #1

Subject	web.mmebvba.com Fingerprint SHA256: 9e7276cb84903692044a0e1f9b64d1426869813b55b28167913b7e49e778f87e Pin SHA256: moiIG7Pck7rm7Q7pJpb+auqA9cuCc0eOAxVrTFBhY0M=
Common names	web.mmebvba.com
Alternative names	- INVALID
Serial Number	00ba5e79e0c2f743cb
Valid from	Mon, 25 May 2015 09:07:54 UTC
Valid until	Thu, 22 May 2025 09:07:54 UTC (expired 9 months and 19 days ago) EXPIRED
Key	RSA 2048 bits (e 65537)
Weak key (Debian)	No
Issuer	web.mmebvba.com Self-signed
Signature algorithm	SHA256withRSA
Extended Validation	No
Certificate Transparency	No
OCSP Must Staple	No
Revocation information	None
DNS CAA	No (more info)
Trusted	No NOT TRUSTED (Why?) Mozilla Apple Android Java Windows

Additional Certificates (if supplied)

Certificates provided	1 (712 bytes)
Chain issues	None

Recommendation

1. **Use Trusted Certificate Authorities:** Digital certificates should only be accepted from well-known and trusted Certificate Authorities (CAs). This ensures that the communication is established with legitimate and verified servers.

2. **Verify Certificate Chains:** The system should validate the complete certificate chain presented by the remote server. This includes verifying intermediate certificates and ensuring that the certificate originates from a trusted authority.

3. **Check Certificate Revocation Status:** It is important to verify whether a certificate has been revoked. This can be done using mechanisms such as Certificate Revocation Lists (CRL) or the Online Certificate Status Protocol (OCSP).
4. **Implement Certificate Pinning:** Certificate pinning ensures that communication is established only with a specific trusted certificate or certificate authority. This reduces the risk of attackers using fraudulent certificates.
5. **Keep Software Updated:** All security protocols, applications, and server software should be kept up to date. Regular updates help fix newly discovered vulnerabilities and improve system security.

---

3) Vulnerability Name: Missing X-Frame-Options Header

CWE: CWE-1021 – Improper Restriction of Rendered UI Layers or Frames
OWASP Category: A04:2021 – Insecure Design

Description

The X-Frame-Options HTTP response header is used to control whether a webpage can be embedded inside frames, iframes, or other HTML elements. When this header is missing, attackers may exploit the application through **clickjacking attacks**, where users are tricked into interacting with hidden or malicious elements embedded within another webpage.

Business Impact

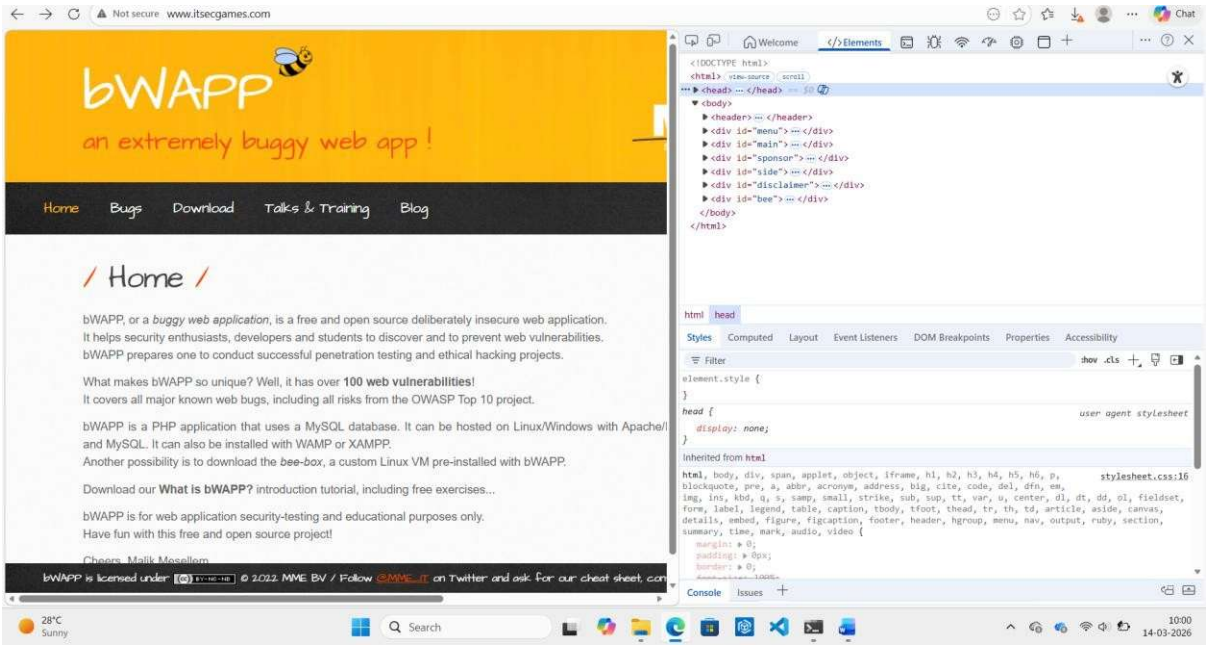
The absence of the X-Frame-Options header allows attackers to manipulate the user interface and deceive users into performing unintended actions. This may result in unauthorized transactions, data exposure, fraudulent activities, and loss of user trust. Such attacks can also damage the organization's reputation and lead to financial losses.

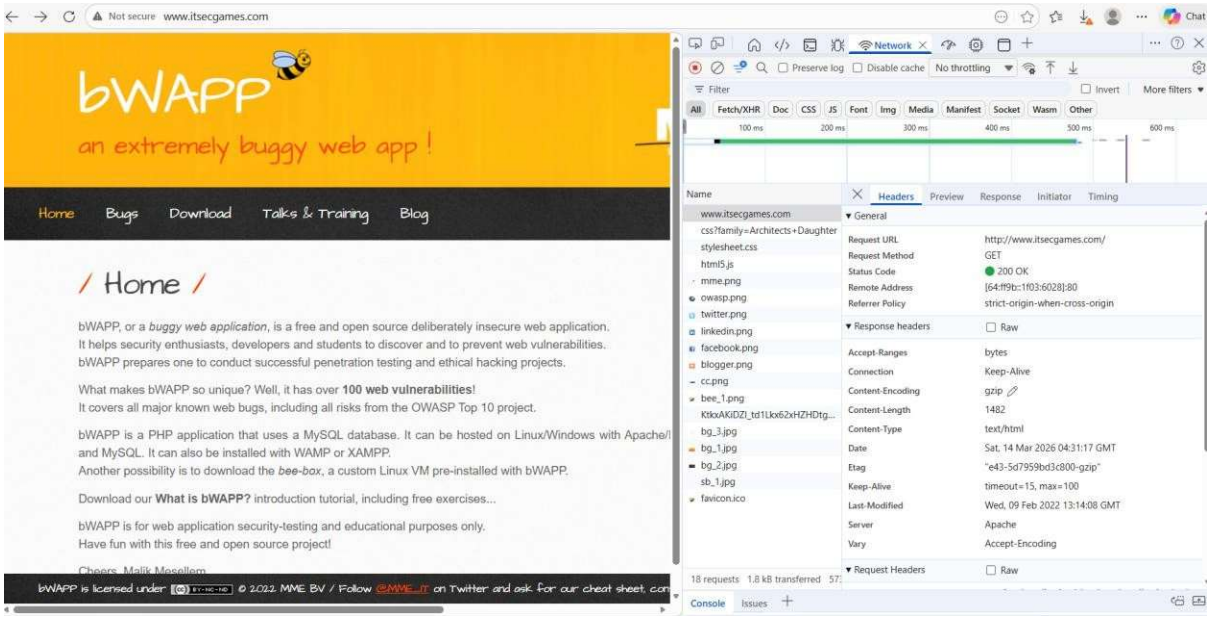
Vulnerability Path

Practice Website:http://www.itsecgames.com/

Steps to Reproduce

1. Open the practice website **http://www.itsecgames.com/** in a web browser.
2. Open the browser's **Developer Tools** and navigate to the **Network** tab.
3. Refresh the webpage by pressing **F5** or **Ctrl + R** to start capturing network requests.
4. Select the **HTML document request** from the captured network traffic.
5. In the **Response Headers** section, check whether the **X-Frame-Options** header is present.
6. If the **X-Frame-Options header is missing**, it indicates that the website does not have protection against embedding its content within an iframe on other domains. This absence may allow attackers to perform **clickjacking attacks**, where users are tricked into interacting with hidden or malicious interface elements.





Recommendation

1. **Implement Proper UI Layer Access Controls:**
   Organizations should enforce strict access control mechanisms to restrict unauthorized access to user interface layers or frames. Access permissions must be granted based on user roles and privileges.

2. **Use Strong Session and Authentication Management:**
   Proper session management and secure authentication mechanisms should be implemented to ensure that only authorized users can access the application. Multi-factor authentication can further enhance security.

3. **Implement Content Security Policy (CSP):**
   Web applications should configure CSP headers to control the sources from which resources can be loaded. This helps in mitigating frame-based attacks and reduces the risk of malicious content being executed.

4. **Keep Software and Libraries Updated:**
   All application components, frameworks, and libraries should be regularly updated to the latest versions. Security patches released by vendors help protect the system from newly discovered vulnerabilities.

4) Vulnerability Name: X-Content-Type-Options Header Missing

CWE: CWE-918 – Server-Side Request Forgery (SSRF)
OWASP Category: A10:2021 – Server-Side Request Forgery (SSRF)

Description

The **X-Content-Type-Options** header is used to prevent browsers from interpreting files as a different MIME type than what is specified by the server. When this header is missing, browsers may attempt to guess the content type of the response. This behavior can be exploited by attackers to manipulate how the browser processes the data, potentially allowing malicious scripts or unexpected content to be executed.

In certain situations, the absence of proper validation mechanisms may also contribute to security weaknesses such as **Server-Side Request Forgery (SSRF)**, where attackers trick the server into sending requests to unintended internal or external resources.

Business Impact

The absence of the **X-Content-Type-Options** header can create several security risks, including:

- **Data Exposure:** Attackers may manipulate the content type and force the browser to interpret sensitive data as executable content, leading to potential data leakage.
- **Security Risks:** Content type manipulation increases the risk of security breaches, unauthorized access, and exploitation of web application vulnerabilities.
- **Reputation Damage:** Successful exploitation of such vulnerabilities may affect the organization's reputation, user trust, and credibility.

Vulnerability Path

Security Header Test Result:
https://securityheaders.com/?q=http%3A%2F%2Fwww.itsecgames.com%2F&followRedirects=on

Security Report Summary

F

Site:	http://www.itsecgames.com/ - (Scan again over https)
IP Address:	31.3.96.40
Report Time:	14 Mar 2026 04:38:29 UTC
Headers:	✖ Content-Security-Policy ✖ X-Frame-Options ✖ X-Content-Type-Options ✖ Referrer-Policy ✖ Permissions-Policy
Warning:	Grade capped at A, please see warnings below.
Advanced:	Ouch, you should work on your security posture immediately: Start Now

Missing Headers

Content-Security-Policy	Content Security Policy is an effective measure to protect your site from XSS attacks. By whitelisting sources of approved content, you can prevent the browser from loading malicious assets.
X-Frame-Options	X-Frame-Options tells the browser whether you want to allow your site to be framed or not. By preventing a browser from framing your site you can defend against attacks like clickjacking. Recommended value "X-Frame-Options: SAMEORIGIN".
X-Content-Type-Options	X-Content-Type-Options stops a browser from trying to MIME-sniff the content type and forces it to stick with the declared content-type. The only valid value for this header is "X-Content-Type-Options: nosniff".
Referrer-Policy	Referrer Policy is a new header that allows a site to control how much information the browser includes with navigations away from a document and should be set by all sites.
Permissions-Policy	Permissions Policy is a new header that allows a site to control which features and APIs can be used in the browser.

```
- Server: Apache
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.
rg/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the
ontent of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-
ulnerability-scanner/vulnerabilities/missing-content-type-header/
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /com.tgz: Drupal 7 was identified via the x-generator header. See: https://www.drupal.org/proje
t/remove_http_headers
+ /com.tgz: Drupal Link header found with value: <http://31.3.96.40/>; rel="canonical",<http://31
3.96.40/>; rel="shortlink". See: https://www.drupal.org/
+ /: Server may leak inodes via ETags, header found with file /, inode: e43, size: 5d7959bd3c800,
mtime: gzip. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ OPTIONS: Allowed HTTP Methods: OPTIONS, GET, HEAD, POST .
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricting-acce
s-to-iconsreadme/
+ 8075 requests: 0 error(s) and 7 item(s) reported on remote host
```

Recommendation

- **Implement "X-Content-Type-Options" Header:**
  Configure the **X-Content-Type-Options** header with the value **"nosniff"** in the web server or application configuration. This prevents browsers from performing MIME type sniffing and ensures that files are interpreted only as the specified content type.
- **Secure Against SSRF Attacks:**
  To mitigate **Server-Side Request Forgery (SSRF)** vulnerabilities, organizations should implement strict input validation and proper access control mechanisms. Allow-list based URL validation should also be applied to ensure that the server communicates only with trusted internal or external resources.
- **Use Additional Security Headers:**
  Implement other security headers such as **X-Frame-Options** and **Content-Security-Policy (CSP)** to strengthen web application security. These headers help protect against multiple web-based attacks including clickjacking and malicious script execution.

Although the **X-Content-Type-Options** header is not a direct mitigation for SSRF vulnerabilities, it acts as an important security control that helps prevent content-type based attacks that could be leveraged in SSRF exploitation scenarios.

5) Vulnerability Name: Drupal Ajax API Shows JSONP Not Disabled by Default

CWE: CWE-937, CWE-1035 – OWASP Top Ten 2013/2017 Category A9 (Using Components with Known Vulnerabilities)
OWASP Category: A06:2021 – Vulnerable and Outdated Components

Description

This vulnerability occurs when a web application uses outdated or insecure software components such as libraries, frameworks, plugins, or APIs. In this case, the **Drupal Ajax API** allows **JSONP functionality** that is not disabled by default. JSONP can introduce security risks if not properly configured, as it may allow unauthorized cross-domain data access.

Outdated components often contain publicly documented vulnerabilities that attackers can exploit to compromise the security of the application. When these components are not updated

or properly secured, they can become entry points for attackers to manipulate application behavior.

Business Impact

The presence of vulnerable or outdated components can create serious security risks for an organization. Attackers may exploit these vulnerabilities to gain unauthorized access to the system, inject malicious code, steal sensitive data, or disrupt normal operations.

Such security breaches can lead to **data leakage, financial loss, regulatory penalties, and damage to the organization's reputation**. In severe cases, organizations may also face legal consequences due to failure in maintaining proper security standards.

5) Vulnerability Name: Drupal Ajax API Shows JSONP Not Disabled by Default

CWE: CWE-937, CWE-1035 – Using Components with Known Vulnerabilities **OWASP Category:** A06:2021 – Vulnerable and Outdated Components

Description

This vulnerability occurs when a web application uses outdated or insecure components such as libraries, frameworks, plugins, or APIs. In this case, the **Drupal Ajax API** allows JSONP functionality that is not disabled by default. If not properly configured, JSONP may allow unauthorized cross-domain access and increase the attack surface of the application.

Business Impact

Using vulnerable or outdated components exposes the application to known security threats. Attackers may exploit these weaknesses to gain unauthorized access, inject malicious code, or steal sensitive data. Such attacks may result in **data breaches, financial losses, reputational damage, and legal consequences** for the organization.

Vulnerability Path

https://www.shodan.io/host/31.3.96.40

Steps to Reproduce

1. Access the target host using a vulnerability scanning tool or browser.
2. Identify the Drupal version and enabled modules used in the web application.

3. Analyze the Ajax API responses and check whether JSONP functionality is enabled by default.
4. Verify if the application is using outdated components with known vulnerabilities.

Recommendation

1. **Regular Updates:** Keep all components, libraries, and dependencies updated with the latest security patches.
2. **Vulnerability Scanning:** Conduct periodic vulnerability scans to identify security issues in third-party components.
3. **Dependency Management:** Implement proper monitoring and management of third-party dependencies.
4. **Security Testing:** Perform regular penetration testing and security assessments.
5. **Change Management:** Maintain a proper change control process to ensure new components do not introduce vulnerabilities.

6) Vulnerability Name: Drupal API Vulnerability

CWE: CWE-20 – Improper Input Validation
OWASP Category: A03:2021 – Injection

Description

This vulnerability occurs when an application receives input from users but fails to properly validate or sanitize the input before processing it. Improper input validation may allow attackers to inject malicious data into the application, potentially leading to **SQL injection or other injection attacks**.

Business Impact

A successful injection attack may allow attackers to gain unauthorized access to sensitive information stored in the database. This can lead to **data theft, financial losses, service disruption, and reputational damage**. In some cases, attackers may also manipulate or delete important data, affecting the integrity and availability of the system.

Vulnerability Path

https://www.shodan.io/host/31.3.96.40

Steps to Reproduce

1. Access the vulnerable API endpoint of the application.
2. Provide specially crafted input values in request parameters.
3. Observe whether the application properly validates or sanitizes the input.
4. If the input is processed directly without validation, it indicates a potential injection vulnerability.

Recommendation

1. **Input Validation:** Validate and sanitize all user inputs before processing them.
2. **Parameterized Queries:** Use prepared statements to separate SQL queries from user input.
3. **Escaping User Input:** Escape special characters if parameterized queries are not used.
4. **Least Privilege Principle:** Restrict database permissions to minimize potential damage.
5. **Web Application Firewall (WAF):** Deploy WAF to detect and block malicious injection attempts.
6. **Regular Security Testing:** Conduct regular vulnerability assessments and penetration testing.

7) Vulnerability Name: Improper API Filtration

CWE: CWE-79 – Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting)
OWASP Category: A07:2017 – Cross-Site Scripting (XSS)

Description

This vulnerability occurs when user-controlled input is not properly sanitized before being displayed in a web page. Attackers can exploit this weakness by injecting malicious scripts into web pages, which are then executed in the browser of other users.

Business Impact

Cross-Site Scripting (XSS) attacks can compromise user accounts, steal sensitive information such as login credentials, and manipulate web page content. This may lead to **data breaches, unauthorized access, loss of user trust, and reputational damage** for the organization.

Vulnerability Path

https://www.shodan.io/host/31.3.96.40

Steps to Reproduce

1. Identify input fields or API parameters in the application.
2. Insert a test script such as <script>alert("XSS")</script> into the input field.
3. Submit the request and observe the application response.
4. If the script executes in the browser, the application is vulnerable to XSS.

Recommendation

1. **Update and Patch:** Keep Drupal and related modules updated with the latest security patches.
2. **Input Validation:** Validate and sanitize user inputs to prevent malicious script injection.
3. **Content Security Policy (CSP):** Implement CSP headers to restrict content sources.
4. **Secure Coding Practices:** Train developers to follow secure coding guidelines.
5. **Security Scanning:** Perform regular security scanning and penetration testing.

8) Vulnerability Name: Denial of Service (DoS)

CWE: CWE-400 – Uncontrolled Resource Consumption

Description

A **Denial of Service (DoS)** attack attempts to make a system, server, or network unavailable to legitimate users by overwhelming it with excessive requests. When the system receives an

unusually large number of requests, it may consume excessive CPU, memory, or network bandwidth, causing the service to slow down or completely stop functioning.

Business Impact

If exploited successfully, a DoS attack can cause major disruptions to business operations. The affected system may become slow or completely unavailable, preventing legitimate users from accessing services. This may result in **financial losses, productivity disruption, service downtime, and reputational damage**. In severe cases, the attack may also lead to system crashes or data loss.

9) Vulnerability Name: TLS Version 1.0 Protocol Detection

CWE: CWE-327 – Use of a Broken or Risky Cryptographic Algorithm
OWASP Category: A02:2021 – Cryptographic Failures

Description

The system supports **TLS version 1.0**, which is considered outdated and insecure. TLS 1.0 lacks modern cryptographic protections and is vulnerable to several security attacks. Attackers may exploit weaknesses in this protocol to intercept or manipulate encrypted communications between users and the server.

Business Impact

The use of TLS 1.0 can expose sensitive information such as login credentials, personal data, and payment information. Attackers may capture or decrypt encrypted traffic. This can lead to data breaches, loss of customer trust, legal consequences, and financial damage to the organization.

Steps to Reproduce

1. Open a terminal or command prompt.
2. Run the following command using **OpenSSL**:

openssl s_client -connect 31.3.96.40:443 -tls1

3. If the connection is successful, it indicates that the server supports TLS 1.0.

```
GET /web.config HTTP/1.1
Host: web.mmebvba.com
Accept-Charset: iso-8859-1,utf-8;q=0.9,*;q=0.1
Accept-Language: en
Connection: Keep-Alive
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 5.1; Trident/4.0)
Pragma: no-cache
Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, image/png, */*



This produced the following truncated output (limited to 5 lines) :
------------------------------ snip ------------------------------
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
<system.webServer>
<!-- Don't show directory listings for URLs which map to a directory. -->
<directoryBrowse enabled="false" />
[...]
```

Recommendation

1. **Upgrade TLS Version**
   Upgrade to **TLS 1.2 or TLS 1.3** for stronger encryption.
2. **Update SSL/TLS Configuration**
   Ensure the server configuration follows modern security standards.
3. **Use Strong Cipher Suites**
   Disable weak ciphers and enable strong encryption algorithms.
4. **Compliance Checks**
   Ensure compliance with industry standards such as **GDPR** and **PCI-DSS**.

10) Vulnerability Name: web.config File Information Disclosure

CWE: CWE-497 – Exposure of Sensitive System Information
OWASP Category: Sensitive Data Exposure

Description

The web server exposes the **web.config file**, which may contain sensitive configuration information. An unauthenticated attacker may access this file using a simple HTTP request and retrieve confidential information.

Business Impact

If attackers gain access to the **web.config** file, they may obtain:

- Database connection strings
- Application configuration details
- Encryption keys
- System settings

This could result in unauthorized system access, data breaches, financial losses, and reputational damage.

Recommendation

- Implement strict **access controls** for configuration files.
- Prevent direct access to the **web.config file** via the web server.
- Encrypt sensitive information stored in configuration files.
- Regularly review and update configuration files.
- Remove unnecessary sensitive data from the configuration file.

Vulnerability: SWEET32 (Medium Strength Cipher Suites Supported)

CWE: CWE-200 – Exposure of Sensitive Information
OWASP Category: Broken Access Control

Description

The server supports SSL/TLS cipher suites that provide **medium-strength encryption**, particularly those using **3DES encryption** with 64-bit block sizes. These cipher suites are vulnerable to the **SWEET32 attack**, which exploits collisions in encrypted data blocks.

Business Impact

If exploited, attackers may intercept or manipulate encrypted communications. This could allow unauthorized access to confidential data and compromise the security of user sessions. Organizations using vulnerable cipher suites risk data breaches, legal issues, and damage to their reputation.

Recommendation

- Disable **3DES cipher suites**.
- Enable **modern cipher suites** such as AES-GCM.
- Update SSL/TLS configurations to support secure encryption algorithms.
- Regularly scan the system using vulnerability scanners such as **Nessus**.

Recommendation:

● Implement proper access controls to restrict access to the web.config file

. ● Prevent directory traversal and ensure the web.config file cannot be directly accessed through the web server.

● Encrypt sensitive data within the configuration file

. ● Regularly review and update the web.config file, removing any unnecessary sensitive information

. ● Ensure that the web.config file is properly secured and not accessible to unauthorized users. Main Website: vtop.vit.ac.in 1) Vulnerability Name: SL Medium Strength Cipher Suites Supported (SWEET32) CWE-200: Exposure of Sensitive Information to an Unauthorised Actor OWASP Category: Broken access control Description: The remote host supports the use of SSL ciphers that offer medium strength encryption. Nessus regards medium strength as any encryption that uses key lengths at least 64 bits and less than 112 bits, or else that uses the 3DES encryption suite. Note that it is considerably easier to circumvent medium strength encryption if the attacker is on the same physical network. Business Impact: The vulnerability known as SWEET32 (Short for Sweet 32 Birthday Attack) relates to the continued use of legacy cryptographic cipher suites with 64-bit block sizes in TLS and SSL protocols. These suites are susceptible to birthday attacks, which exploit the probability of two blocks colliding after a certain number of transactions. If an organisation's systems are vulnerable to SWEET32, it means they're potentially at risk of having encrypted communications intercepted or manipulated. This could lead to unauthorised access, data theft, or even the injection of malicious code. As a result, businesses face a significant risk to the confidentiality and integrity of their sensitive information. Remediation involves updating systems to support more secure cipher suites with larger block sizes, which would effectively mitigate this vulnerability and enhance the overall security posture. Failure to address SWEET32 could leave an organisation

exposed to increasingly sophisticated attacks and legal consequences due to potential data breaches.

Vulnerability Path : The Sweet32 vulnerability when detected with a vulnerability scanner will report it as a CVSS 7.5.

HIGH SSL Medium Strength Cipher Suites Supported (SWEET32)

Description

The remote host supports the use of SSL ciphers that offer medium strength encryption. Nessus regards medium strength as any encryption that uses key lengths at least 64 bits and less than 112 bits, or else that uses the 3DES encryption suite.

Note that it is considerably easier to circumvent medium strength encryption if the attacker is on the same physical network.

Solution

Reconfigure the affected application if possible to avoid use of medium strength ciphers.

See Also

https://www.openssl.org/blog/blog/2016/08/24/sweet32/
https://sweet32.info

Output

```
Medium Strength Ciphers (> 64-bit and < 112-bit key, or 3DES)

  Name                      Code           KEX        Auth     Encryption            MAC
  ----------------------    ----------     ---        ----     ---------------------  ---
  EDH-RSA-DES-CBC3-SHA      0x00, 0x16     DH         RSA      3DES-CBC(168)         SHA1
  ECDHE-RSA-DES-CBC3-SHA    0xC0, 0x12     ECDH       RSA      3DES-CBC(168)         SHA1
  DES-CBC3-SHA              0x00, 0x0A     RSA        RSA      3DES-CBC(168)         SHA1

The fields above are :

  {Tenable ciphername}
  {Cipher ID code}
  Kex={key exchange}
  Auth={authentication}
  Encrypt={symmetric encryption method}
  MAC={message authentication code}
  {export flag}
```

To see debug logs, please visit individual host

Port	Hosts
443 / tcp / www	136.233.9.22

Plugin Details

Severity:	High
ID:	42873
Version:	1.21
Type:	remote
Family:	General
Published:	November 23, 2009
Modified:	February 3, 2021

VPR Key Drivers

Threat Recency: No recorded events
Threat Intensity: Very Low
Exploit Code Maturity: PoC
Age of Vuln: 730 days +
Product Coverage: High
CVSSV3 Impact Score: 3.6
Threat Sources: No recorded events

Risk Information

Vulnerability Priority Rating (VPR): 6.1
Risk Factor: Medium
CVSS v3.0 Base Score 7.5
CVSS v3.0 Vector:
CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CVSS v2.0 Base Score: 5.0
CVSS v2.0 Vector:
CVSS2#AV:N/AC:L/Au:N/C:P/I:N/A:N

Vulnerability Information

Vulnerability: SWEET32 (Medium Strength Cipher Suites Supported)

CWE: CWE-200 – Exposure of Sensitive Information to an Unauthorised Actor
OWASP Category: Broken Access Control

Description

The remote server supports SSL/TLS cipher suites that provide **medium-strength encryption**, particularly those using **3DES encryption with 64-bit block sizes**. These cipher suites are vulnerable to the **SWEET32 attack**, which exploits collisions in encrypted data blocks during long encrypted sessions.

Business Impact

If exploited, attackers may intercept encrypted communications between the user and the server. This could allow attackers to obtain sensitive information such as login credentials, session cookies, or confidential data. Such vulnerabilities may lead to data breaches, legal liabilities, financial losses, and reputational damage for the organization.

Recommendation

Organizations should take several proactive steps to mitigate the SWEET32 vulnerability:

1. **Disable Weak Cipher Suites**
   Disable all cipher suites that use **64-bit block sizes**, particularly **3DES encryption**.
2. **Enable Modern Encryption Standards**
   Configure systems to use strong encryption algorithms such as **AES-GCM**.
3. **Update Cryptographic Libraries**
   Regularly update web servers, cryptographic libraries, and network devices with the latest security patches.
4. **Use Forward Secrecy**
   Implement forward secrecy protocols to ensure encrypted sessions remain secure even if encryption keys are compromised.
5. **Perform Regular Security Audits**
   Conduct regular vulnerability scans using security tools such as **Nessus**.
6. **Deploy Intrusion Detection Systems**
   Monitor suspicious network traffic using IDS/IPS systems.
7. **Follow Security Best Practices**
   Stay updated with emerging cybersecurity threats and continuously improve cryptographic configurations.

Vulnerability 2: Cookie SERVERID Created Without Secure Flag

CWE: CWE-79 – Improper Neutralization of Input During Web Page Generation
OWASP Category: A07:2017 – Cross-Site Scripting (XSS)

Description

The vulnerability occurs when the **SERVERID cookie** is created without the **Secure flag**. This means the cookie may be transmitted over unencrypted HTTP connections. Attackers may exploit this vulnerability to intercept session cookies or inject malicious scripts into the web application.

In some cases, attackers may inject malicious code into a website, which can then be executed by users who visit the affected page. This type of attack is commonly known as **Cross-Site Scripting (XSS)**.

Business Impact

Cross-Site Scripting attacks can lead to severe consequences, including:

- Theft of user session cookies
- Unauthorized access to user accounts
- Data breaches and exposure of sensitive information
- Loss of customer trust and reputational damage
- Financial losses due to fraud or recovery costs

If customers lose confidence in the platform's security, they may stop using the service, causing long-term damage to the organization.

Vulnerability Path

Security analysis can be performed using online tools such as:

https://securityheaders.com/

Example test URL:

q=http%3A%2F%2Fvtop.vit.ac.in%2F&followRedirects=on

Vulnerability 3: X-Content-Type-Options Header Missing

CWE: CWE-918 – Server-Side Request Forgery (SSRF)
OWASP Category: A10:2021 – Server-Side Request Forgery (SSRF)

Description

The absence of the **X-Content-Type-Options** HTTP header introduces a potential security vulnerability in web applications. This header instructs web browsers to strictly follow the MIME type specified in the **Content-Type** header instead of attempting to guess the content type.

When this header is missing, browsers may perform **MIME sniffing**, where the browser tries to determine the file type on its own. This behavior can be exploited by attackers to disguise malicious files (such as scripts or executable files) as safe content types.

As a result, malicious content may be executed in the user's browser, leading to attacks such as **Cross-Site Scripting (XSS)** or other web-based exploits.

Implementing the **X-Content-Type-Options: nosniff** header prevents browsers from interpreting files incorrectly and improves the overall security of the web application.

Business Impact

If exploited, this vulnerability may result in:

- Exposure of sensitive data if attackers force the browser to render sensitive information as executable content.
- Increased security risks through content-type manipulation.
- Potential unauthorized access or data leaks.
- Damage to the organization's brand reputation and customer trust.

Vulnerability Path

Security testing can be performed using online tools such as:

https://securityheaders.com/?q=http%3A%2F%2Fvtop.vit.ac.in%2F&followRedirects=on

Recommendation

To mitigate this vulnerability, the following security practices should be implemented:

1. **Implement X-Content-Type-Options Header**
   Configure the header as:X-Content-Type-Options: nosniff
2. **Secure Against SSRF**
   Apply strict input validation and allow-list based URL validation to prevent unauthorized server requests.
3. **Use Additional Security Headers**
   Implement headers such as:
   - X-Frame-Options

- Content-Security-Policy (CSP)

4. **Regular Security Audits**
   Conduct frequent security testing and vulnerability scanning to identify and fix potential weaknesses.

Vulnerability 4: Denial of Service (DoS)

CWE: CWE-400 – Uncontrolled Resource Consumption

Description

A **Denial of Service (DoS)** attack aims to make a system, server, or application unavailable to legitimate users by overwhelming it with excessive requests or exploiting vulnerabilities.

Attackers may send a very large number of requests to a server, consuming critical system resources such as CPU, memory, or network bandwidth. As a result, the server may become slow or completely unresponsive.

DoS attacks can also exploit programming or logic vulnerabilities in the application, causing system crashes or service interruptions. In some cases, attackers may execute arbitrary code during the attack to gain unauthorized access or compromise the system.

These attacks severely degrade the quality of service experienced by legitimate users by introducing delays, packet loss, and service interruptions.

Business Impact

If successfully exploited, a DoS attack may lead to:

- Significant system slowdowns or complete service outages.
- Depletion of system resources such as CPU, memory, and network bandwidth.
- Loss of business productivity and operational disruptions.
- Potential data loss or system crashes.
- Financial losses and reputational damage.
- Loss of customer trust and service reliability.

Organizations may also face regulatory compliance issues if service availability requirements are not met.

Steps to Reproduce

1. Identify open ports on the target system (e.g., port **443**).
2. Attempt to exploit vulnerabilities associated with the open port.
3. Launch repeated requests or malicious traffic to overload the server.
4. Observe service degradation or denial of access to legitimate users.

Example attack types may include:

- SQL Injection attacks
- Cross-Site Request Forgery (CSRF)
- Distributed Denial of Service (DDoS) attacks

Recommendation

To mitigate Denial of Service attacks, organizations should implement the following measures:

1. **Deploy Web Application Firewalls (WAF)** to filter malicious traffic.
2. **Implement Rate Limiting** to restrict the number of requests from a single IP address.
3. **Use Load Balancing** to distribute traffic across multiple servers.
4. **Monitor Network Traffic** and set alerts for unusual spikes in activity.
5. **Update Systems Regularly** with the latest security patches.
6. **Deploy Intrusion Detection Systems (IDS)** to detect suspicious behavior.
7. **Implement DDoS Protection Services** to absorb and mitigate large-scale attacks.

```
C:\Users\ELCOT>nmap --top-ports 65536 31.3.96.40
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-14 11:00 India Standard Time
Nmap scan report for web.mmebvba.com (31.3.96.40)
Host is up (0.094s latency).
Not shown: 8374 filtered tcp ports (no-response)
PORT    STATE SERVICE
22/tcp  open  ssh
80/tcp  open  http
443/tcp open  https

Nmap done: 1 IP address (1 host up) scanned in 146.76 seconds
```

Recommendation: employ a robust Web Application Firewall (WAF) capable of filtering and mitigating suspicious traffic targeting port 80. Configure it to block excessive requests or patterns indicative of an attack. Utilise load balancing techniques to distribute traffic across multiple servers, ensuring no single server becomes overwhelmed. Additionally, implement rate limiting and throttling mechanisms to restrict the number of requests a single IP address can make within a defined time frame. Regularly monitor traffic patterns and set up alerts to detect unusual spikes in activity, enabling rapid response to potential attacks. Finally, keep software and systems up-to-date with the latest security patches to address known vulnerabilities that attackers may exploit.

5) Vulnerability Name: OS Identification CWE-200: Exposure of Sensitive Information to an Unauthorised Actor OWASP Category: A02:2021-Cryptographic Failures Description: Using a combination of remote probes (e.g., TCP/IP, SMB, HTTP, NTP, SNMP, etc.), it is possible to guess the name of the remote operating system in use. It is also possible sometimes to guess the version of the operating system. Business Impact: If sensitive information is improperly handled or inadequately protected, it can lead to financial losses, regulatory penalties, and damage to reputation. Customers may lose trust in the organization's ability to safeguard their data, potentially resulting in a loss of business and market share. Legal consequences and compliance issues may arise, leading to additional costs. Moreover, intellectual property theft or exposure can compromise a company's competitive edge. Remediation efforts, including implementing stronger security measures and compensating affected parties, can also entail substantial expenses. Overall, CWE-200 highlights the critical importance of robust data protection measures to safeguard an organization's financial health, reputation, and long-term viability.

Steps to Reproduce: use nmap -O TARGET to enable os detection

Recommendation

To protect sensitive information and reduce the risk of security breaches, organizations should implement strong security measures. Data should be protected using robust encryption protocols both **in transit and at rest**. Additionally, strict access control mechanisms and strong authentication methods such as **multi-factor authentication (MFA)** should be enforced to ensure that only authorized users can access sensitive resources.

Regular **vulnerability assessments and penetration testing** should be conducted to identify and fix potential security weaknesses within the system. Organizations must also focus on **security awareness training** for employees to educate them about safe data handling practices and how to recognize possible cyber threats.

Furthermore, the use of **data masking techniques** in non-production environments can help prevent the exposure of sensitive information during development and testing activities. Implementing advanced **Data Loss Prevention (DLP)** solutions can further strengthen the organization's security posture by monitoring and preventing unauthorized data transfers.

Compliance with relevant **data protection laws and regulations** is also essential for maintaining security and trust. Finally, organizations should establish a well-defined **incident**

response plan to ensure that security incidents are handled quickly and effectively to minimize potential damage.

Stage 1

Stage 1 of this project focuses on **web application security testing** and vulnerability identification. The analysis is performed by referring to the widely recognized **OWASP Top 10 security risks**.

The **OWASP Top 10** is a globally accepted standard that identifies the most critical security risks affecting web applications. It provides guidance for developers, security professionals, and organizations to understand common vulnerabilities and implement appropriate security controls.

By analysing web applications based on the OWASP Top 10 framework, security testers can detect common issues such as **broken access control, injection attacks, insecure design, security misconfiguration, and vulnerable components**. Understanding these risks helps organizations improve the security posture of their applications and protect sensitive data from potential cyber threats.

Stage 2

Overview

Nessus vulnerability scanner is a vulnerability scanning tool developed by Tenable, Inc.. It is widely used by security professionals to identify vulnerabilities in systems, networks, and web applications. Nessus operates as a **remote vulnerability scanner**, which means it does not need to be installed on the target system in order to perform security scans.

Nessus works by sending different types of probes to the target host to identify potential vulnerabilities. These probes may include **network traffic analysis, operating system commands, and application-level requests**.

The tool uses a **plugin-based architecture**, where each plugin is designed to detect a specific vulnerability. Thousands of plugins are available, covering vulnerabilities in **operating systems, web applications, network devices, and software services**.

During a scan, Nessus first performs **host discovery** to identify the operating system and services running on the target machine. After identifying the services, Nessus executes multiple plugins to test the system for known vulnerabilities. Once the scan is completed, Nessus generates a **detailed vulnerability report** that includes severity levels, descriptions, reproduction steps, and remediation recommendations.

How Nessus Works

The vulnerability scanning process using Nessus can be explained in the following steps:

1. Nessus connects to the target system using network protocols such as **TCP, UDP, and ICMP**.
2. The scanner sends multiple probes to detect the **operating system and active services** running on the host.
3. Nessus executes various **security plugins** to check for known vulnerabilities.
4. After the scan is completed, Nessus generates a **comprehensive vulnerability report** containing the severity level, vulnerability description, and recommended solutions.

Benefits of Using Nessus

- **Comprehensive Vulnerability Coverage:** Nessus supports thousands of plugins that detect vulnerabilities in operating systems, applications, and network devices.
- **User-Friendly Interface:** The platform provides an easy-to-use interface for configuring and running vulnerability scans.
- **Detailed Reporting:** Nessus generates reports containing vulnerability severity, reproduction steps, and remediation guidelines.
- **Scalability:** It can scan individual systems, entire networks, or web applications.
- **Flexibility:** Scans can be executed manually or scheduled automatically.

Because of these features, Nessus is considered an essential tool for improving the **security posture of modern IT infrastructures**.

Target Details

Target Website:
http://testfire.net/

Target IP Address:
65.61.137.117

Stage – 3

Report:

Title: Implementing Zero Trust Network Access (ZTNA) with Zscaler: Securing User-to-Application Connectivity

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a modern IT security framework that provides secure remote access to an organization's internal applications and data based on clearly defined access control policies. Unlike traditional Virtual Private Networks (VPNs), which provide users with broad access to the entire network, ZTNA operates on the principle of **"Never Trust, Always Verify."**

Under this model, users are granted access only to the specific applications they require, and only after their identity and device security posture have been verified. This ensures that unauthorized users or compromised devices cannot gain access to sensitive systems.

ZTNA plays a crucial role in modern cybersecurity architecture by replacing the traditional **"castle-and-moat" security model**, where internal networks are trusted and external networks are not. Instead, ZTNA hides internal applications from the public internet and establishes **secure one-to-one encrypted connections** between users and applications. This significantly reduces the attack surface and prevents threats such as **lateral movement within the network**.

The ZTNA Access Cycle

The ZTNA access process follows a continuous cycle that verifies identity, enforces policy, and establishes secure connections for every request.

1. User Request

A user attempts to access a specific internal application from any location such as home, office, or a public network.

2. Identity and Device Verification

The ZTNA system verifies the user's identity using authentication methods such as **Multi-Factor Authentication (MFA)**. At the same time, the device posture is checked to ensure it meets security requirements such as updated antivirus software and firewall protection.

3. Policy Enforcement

Access policies are evaluated to determine whether the user and device are authorized to access the requested application at that particular time and location.

4. Secure Connection Establishment

If the policy conditions are satisfied, the system establishes a secure encrypted micro-tunnel between the user’s device and the target application.

5. Continuous Monitoring

The connection is continuously monitored for suspicious behavior. If abnormal activity is detected, the connection can be terminated immediately.

6. Logging and Analytics

All access attempts, whether successful or unsuccessful, are logged for auditing, monitoring, and threat analysis.

Zscaler Private Access (ZPA)

Zscaler provides a cloud-native security platform that enables organizations to implement Zero Trust Network Access effectively.

Zscaler Private Access (ZPA) is a cloud-based service that provides secure access to private applications without placing users directly on the corporate network.

Instead of exposing applications to the internet, ZPA connects users securely to applications through the Zscaler cloud. The platform collects security information from multiple sources such as identity providers, endpoint security tools, and internal applications.

Data Sources Used by Zscaler

1. **Identity Providers (IdP)** – Systems such as Microsoft Azure AD authenticate users and verify identity.
2. **Device Posture Partners** – Endpoint security tools check device health and compliance.
3. **Business Applications** – Internal systems such as ERP servers, research databases, and administrative portals.
4. **Zscaler App Connectors** – Lightweight connectors deployed inside the data center to securely connect internal applications to the Zscaler cloud.

Zscaler ZTNA Operational Cycle

The Zscaler implementation of ZTNA follows the steps below:

1. User Authentication

The user attempts to access an internal application and is redirected to the organization's Identity Provider for authentication.

2. Device Security Verification

The endpoint device is checked to ensure it meets required security standards.

3. Policy Evaluation

The Zscaler cloud evaluates the request based on user identity, device status, location, and time.

4. Micro-Tunnel Creation

Once authorized, a secure encrypted micro-tunnel is established between the user device and the application server.

5. Traffic Inspection

Optional security inspection mechanisms such as Data Loss Prevention (DLP) and malware detection can analyze application traffic.

6. Activity Logging

All activity is recorded in the Zscaler portal for monitoring, compliance, and threat detection.

MISP Threat Intelligence Integration

Malware Information Sharing Platform (MISP) can be integrated with Zscaler to enhance threat intelligence capabilities.

Benefits of Integration

1. **Threat Intelligence Feed**
   MISP provides updated lists of malicious domains, IP addresses, and URLs.
2. **Proactive Threat Blocking**
   If a user attempts to access a malicious resource, Zscaler can automatically block the request.
3. **Enhanced Visibility**
   Security teams can analyze Zscaler logs and cross-reference them with MISP threat intelligence data.

College Network Infrastructure

The college network environment typically includes multiple interconnected systems such as:

- Core routers and switches
- Distribution network devices
- Wireless access points
- Firewalls and Intrusion Detection Systems (IDS)
- Internal application servers

- Legacy VPN infrastructure

These components create a large attack surface, particularly when remote access is required for students, faculty, and administrative staff.

Limitations of Traditional VPN

Traditional VPN solutions present several challenges:

- **Over-privileged access** allowing users to access the entire network
- **Risk of lateral movement** if a device is compromised
- **Slow user experience** due to centralized routing
- **Limited scalability** for large user bases

Proposed Deployment of ZTNA in College Network

To deploy Zscaler ZTNA effectively within the college network, the following steps can be followed:

1. Application Discovery

Identify all internal applications including student portals, research servers, and administrative systems.

2. User Segmentation

Define access groups such as students, faculty members, administrative staff, IT personnel, and external collaborators.

3. Deploy Zscaler App Connectors

Install App Connector virtual machines in the college data center to securely connect internal applications to the Zscaler cloud.

4. Identity Integration

Integrate Zscaler with the institution's identity provider such as Azure Active Directory.

5. Access Policy Configuration

Define granular access policies specifying which user groups can access specific applications.

6. Client Connector Deployment

Install the Zscaler Client Connector on user devices to enable secure connectivity.

Conclusion

This project explored multiple aspects of cybersecurity and web security technologies through three major stages.

Stage 1 focused on web application security testing and understanding vulnerabilities within web applications. This stage provided the foundational knowledge required to identify common security weaknesses.

Stage 2 examined vulnerability assessment techniques using the vulnerability scanner Nessus vulnerability scanner. The analysis of vulnerability reports helped identify potential security threats and highlighted the importance of proactive security management.

Stage 3 introduced the concept of Zero Trust Network Access and demonstrated how Zscaler technologies can enhance security by implementing identity-based access control. By replacing traditional VPN models with ZTNA architecture, organizations can significantly reduce security risks, prevent lateral movement attacks, and ensure secure application access.

The study highlights the importance of adopting modern cybersecurity frameworks to protect digital infrastructure in an increasingly connected world.

Future Scope

The future of cybersecurity and web application testing will continue to evolve with emerging technologies and changing threat landscapes.

Key Future Trends

- Increased focus on security testing
- Adoption of Artificial Intelligence and Machine Learning

- Growth of cloud-based testing platforms
- Greater emphasis on automated testing and continuous integration
- Expansion of Zero Trust security frameworks

Future implementations of Zero Trust architectures will integrate advanced AI-driven threat detection, adaptive authentication mechanisms, and automated response systems to provide stronger protection against evolving cyber threats.

Tools Explored

During the project, several cybersecurity tools were used for testing and vulnerability analysis:

- Nmap
- Metasploit Framework
- Nessus vulnerability scanner
- Kali Linux
- OWASP ZAP
- Burp Suite